CSS

Introduction to Cascading Style Sheets

syntax, rule, selector, property, value

Spolufinancováno Evropskou unií

MINISTERSTVO ŠKOLSTVÍ, MLÁDEŽE A TĚLOVÝCHOVY

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

doc. Ing. Radek Burget, Ph.D.

burgetr@vut.cz

Ing. Jiří Hynek, Ph.D.

hynek@vut.cz

Visual Style in HTML

- **HTML 4**
  - contained presentational tags (`<font>`, `<center>`, `<big>`, …)
  - and attributes (`bgcolor`, `align`, `border`, …)

- 🔴 **disadvantages:**
  - cluttered and hard-to-maintain code
  - changing appearance means editing every HTML file → tedious and error-prone
  - document cannot be easily reformatted for other outputs (mobile version, print, readers)
  - poor accessibility → for example, <b> does not tell a reader why text is highlighted
  - **dynamically generated content**
    - program must generate both content and appearance
    - changing appearance requires changing the program

Visual Style in HTML – Example

Separation of Appearance from Content

- **HTML 5**
  - describe the **structure** and **meaning** of content
  - appearance described separately in a **style sheet** (Cascading Style Sheet)

- 🟢 **advantages:**
  - **separation of content and presentation** – HTML handles structure and meaning, CSS handles appearance
  - **easy maintenance** – changing one CSS class updates appearance across all web pages
  - **reusability** – the same styles can be applied to different documents
  - **flexibility** – different styles for different devices (mobile, print, dark mode)
  - **advanced features** – animations, responsive design, variables, grid/flexbox – HTML alone cannot do this

Separation of Appearance from Content – Example

CSS Language

Introduction, Principles

CSS

- CSS = *Cascading Style Sheet*
- Standard language for creating style sheets
  - **Specification** maintained by W3C (World Wide Web Consortium), more readable version: **MDN**

- **Origins (1994 - 1996)** – birth of CSS, first specification (CSS1), basic styles for text and colors
- **Development (1998 - 2000)** – CSS2 introduces positioning, layers, media types
  - ⚠️ inconsistent browser support
- **Modularization and CSS3 (2001 - 2010)** – development split into **modules** (CSS 3)
  - flexbox, animations, advanced selectors, responsive media queries
- **Modern Layouts (2011 - 2020)** – greater focus on responsive design and mobile web
  - 2011 – CSS 2.1 W3C Recommendation – ✅ stable, supported by browsers
  - CSS Grid, custom properties (variables)
- **Present (2021 - today)** – container queries, subgrid, :has() selector, native nesting, Houdini API
  - CSS 3+ modules

Style Sheet

- **sequence of rules** concerning various elements that determine their **properties**
  - e.g. color and font weight of <p> elements
  - typically multiple rules for each element

- sources of style sheets:
  1. author style (prepared by the page creator)
  2. user style (*user stylesheets*) - user's own definitions
  3. browser style (*user agent styles*)

- relevant rules are arranged into a **cascade**
  - fully define the final appearance (properties) of each element

Linking to an HTML Document

- **external**, **internal**, and **inline** style sheets

```html
<head>
  <!-- external style sheet -->
  <link rel="stylesheet" type="text/css" href="style.css">

  <!-- internal style sheet -->
  <style>
  /* CSS language inside HTML document */
  p {
    color: gray;
  }
  </style>
</head>

<body>
  <!-- inline style sheet -->
  <p style="font-weight: bold; font-style: italic;"></p>
</body>
```

- **beware of priorities!** – more on this later…

CSS Syntax

```
/* rule */
selector {
  property: value;
  property: value;
  …
}

/* another rule */
selector {
  property: value;
  property: value;
  …
}

…
```

- **selector** – specifies which elements the rule applies to
- **property** – defines visual (or other) characteristics of the selected elements

Selectors

simple, structured, pseudo-classes, pseudo-selectors

Selector

- an operation that returns a **set of elements**
  - properties are applied to the selected elements
- there are **different types of selectors**
  - simple – e.g. `p`, `.class`, `#id`, `[attribute]`
  - structured – e.g. `div > p`, `a + b`
  - pseudo-classes – `:hover`
  - pseudo-elements – `::before`
- each selector has a certain **specificity** (`id` > `class` > `element`), which determines which style takes precedence
  - will be covered in the next lecture

Overview of CSS Selectors

Universal Selector: *

- applies to **all elements in the document**

```
* {
  border: 4px solid green;
  background-color: #00FF0011;
}
```

```
<body><h1>ITW</h1><H2>ABOUT</H
```

Type Selector: `tag`

- applies to all elements **of the given name**

```
h1 {
  border: 4px solid blue;
  background-color: lightblue;
}

h2 {
  border: 4px solid green;
  background-color: lightgreen;
}

i {
  border: 4px solid red;
  background-color: #ffcdcdff;
}
```

<h1>ITW</h1><H2>ABOUT</H2><p>

Class Selector: `.class`

- applies to all elements **of the given class**
  - multiple elements can share a class, and one element can have multiple classes

```
.box {
  border: 4px solid blue;
  background-color: lightblue;
}

h2.box {
  /* overrides .box */
  border: 4px solid green;
  background-color: lightgreen;
}

/* same as .red */
*.red {
  color: red;
}
```

<h1 class="box">ITW</h1>

<H2>ABOUT

ID Selector: #id

- applies to the unique element **with the given identifier**
  - if there are **incorrectly** multiple, it applies to all occurrences

```
#unique {
  border: 4px solid blue;
  background-color: lightblue;
}

/* generally doesn't make much sens
   since according to the
   HTML specification
   it should be a unique
   element in the document */
i#unique {
  color: red;
}
```

<h1 id="unique">ITW</h1>

<H2>ABOUT

Attribute Selector: `[attribute]`

- applies to all elements whose attributes match the condition

```
[title] {
  color: red;
}

h1[title='Web design'] {
  border: 4px solid blue;
  background-color: lightblue;
}

h2[title*='eb'] {
  border: 4px solid green;
  background-color: lightgreen;
}
```

<h1 title="Web design">ITW</h1>

<H2 TIT

- *= substring, ^= prefix, $= suffix
- ~= substring separated by whitespace
- |= value starts with the string or the string followed by a hyphen

Structured Selectors

- compound selector: **`s1 op s2`**, where op is the following operator

```css
/* descendants */
article p {
  color: red;
}

/* direct children */
article > p {
  border: 4px solid blue;
}

/* following siblings */
div ~ p {
  background-color: lightblue;
}

/* immediately following sibling */
div + p {
  font-style: italic;
  font-weight: bold;
}
```

```html
<article>

 <div class="lectures">

  <p>Web</p>

  <p>HTML</p>

  <p>CSS</p>

 </div>
```

Grouping

- multiple rules with the same properties can be grouped into one rule

```
h1 {
  color: red;
  font-weight: bold;
}

section h2 {
  color: red;
}
```

- grouping for the property `color: red`

```
h1, section h2 {
  color: red;
}

h1 {
  font-weight: bold;
}
```

Pseudo-classes

- special selectors starting with `:`, targeting the state of an element, its position, or logical context:

```
p:nth-of-type(1) {
  border: 4px solid blue;
}

h2:hover {
  color: blue;
}
```

```
<article>

<H2>ABOUT</H2>

<p>Hello! You shall not pass!.</

<div class="lectures">

<p>Web</p>

<p>HTML</p>
```

- more in vertical slides or next lectures…

Pseudo-classes

- **types:**
  - **stateful** (user interaction)
    - `:hover`, `:active`, `:focus`, `:target`, ...
  - **form states**
    - `:enabled`, `:checked`, `:valid`, `:required`, ...
  - **structural** (position in DOM)
    - `:root`, `:first-child`, `:nth-child(n)`, `:first-of-type`, ...
  - **logical**
    - `:not`, `:is`, `:where`, `:has`, ...
  - **others** (language, links, and other special…)
    - `:lang`, `:dir`, `:link`, `:visited`, ...

Pseudo-classes: stateful

```
/* mouse hover */
article *:hover {
  color: blue;
}

/* mouse click and hold */
article *:active {
  color: red;
}

/* active via TAB key */
article *:focus {
  border: 6px solid green;
}

/* clicking a link
   referring to an element */
body *:target {
  border: 4px solid orange;
}
```

```
<a href="#about">About</a>
```

```
<article id="abou
```

```
<H2>ABOUT
```

```
<p>Hello! You sha
```

```
<input type="te
```

Name

```
</input>
```

```
<input type="te
```

Surname

Pseudo-classes: form states

```css
input:required {
  border: 6px solid tomato;
}

input:valid {
  background-color: lightgreen;
}

input:invalid {
  background-color: #FFCCCB;
}

input:disabled + label {
  font-style: italic;
}

input[type='checkbox']:checked
  + label {
  color: red;
}
```

```html
<input type="text" placeholder="Name">
```

Name

Pseudo-classes: structural

```
/* <html element> */
:root { border: 4px solid red; }

p:nth-of-type(1) {
  border: 4px solid blue;
}
p:nth-child(1) {
  background-color: lightblue;
}
```

```
<article>
<H2>ABOUT</H2>
<p>Hello! You shall not pass!.</
<div class="lectures">
<p>Web</p>
<p>HTML</p>
```

- **node order in parent:**
  - `:first-child`, `:last-child`, `:only-child`, `:nth-child(n)`, `:nth-last-child(n)`
- **type order in parent:**
  - `:first-of-type`, `:last-of-type`, `:only-of-type`, `:nth-of-type(n)`, `:nth-last-of-type(n)`

Pseudo-classes: logical

```
/* all elements containing h2 */
*:has(> h2) {
  border: 4px solid blue;
}

/* all elements except article,
   containing p as a direct child */
:not(article):has(> p) {
  border: 4px solid green;
}

/* all h2 and p elements,
   that are direct children of article */
article > :is(h2, p) {
  color: red;
}

/* all elements except p and div,
   that are direct children of article or div */
:where(article, div) > :not(:is(p, div)) {
  border: 4px solid orange;
}
```

```
<article>

  <H2>ABOUT</H2>

  <p>Hello! You shall not pass!.</p>

  <div class="lectures">
    <p>Web</p>
    <p>HTML</p>
```

Pseudo-classes: other specific…

- `:lang()` – language-based selection (attribute `lang`)
- `:dir(rtl)` – elements with right-to-left text direction (e.g. Arabic, Hebrew)
- `:link` – unvisited links
- `:visited` – links the user has visited
- `:any-link` – all links (`:link` and `:visited`)
- and many more…

- more in **MDN**…

Pseudo-elements

- part of CSS selectors (not a selector itself) starting with `::`, allowing you to target a specific part of an element, even if there is no separate tag for it in HTML:

```css
p::first-line {
  font-weight: bold;
  color: blue;
}

p::first-letter {
  font-size: 3em;
  color: red;
  float: left;
  margin-right: 5px;
}

p::selection {
  background: yellow;
  color: black;
}
```

```html
<p>You shall not pass!. One ring to
```

- more in vertical slides or next lectures…

Pseudo-elements

- **types:**
  - **text** (targeting part of text)
    - `::first-line`, `::first-letter`, `::selection`, ...
  - **input and form** (specific for `<input>`, `<textarea>`, and forms)
    - `::placeholder`, `::file-selector-button`, `::cue`, ...
  - **content** (for adding or manipulating content)
    - `::before`, `::after`
  - **other special**
    - `::marker` – list item marker styling `<li>`
    - `::cue` – subtitle styling in `<track>` for `<video>`
    - ...

Pseudo-elements: text

```
/* First line of text will be bold and blue */
p::first-line {
  font-weight: bold;
  color: blue;
}

/* First letter will be huge and red */
p::first-letter {
  font-size: 3em;
  color: red;
  float: left;
  margin-right: 5px;
}

/* Text selected by the user will
   have yellow background and black text */
p::selection {
  background: yellow;
  color: black;
}
```

```
<p>You shall not pass!. One ring to
```

Pseudo-elements: input and form

```
input::placeholder {
  color: SandyBrown;
  font-style: italic;
}

/* style the button */
input[type="file"]::file-selector-button {
  background-color: #4CAF50;
  color: white;
  padding: 5px 10px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}

input[type="file"]::file-selector-button:hover {
  background-color: #45a049;
}
```

<H2>PROJECT 1</H2>

Pseudo-elements: content

- possibility to add new elements to the DOM

```
span:not(.code-tag)::before {
  content: '&#x1F9D9;'
}

span:not(.code-tag)::after {
  content: ' ';
  display: inline-block;
  width: 100px;
  height: 100px;
  background-image: url('gandalf.png');
  background-size: contain;
  background-repeat: no-repeat;
  vertical-align: middle;
}
```

```
<span>You shall not pass!.</span>
```

- part of the element (first or last child)
- the rule must always contain the **`content`** property

Property Values

specific, numbers, lengths, sizes, measures, functions, colors

Property Value

- a specific setting that defines **how a property should behave** and what effect it has on rendering an element in the browser
  - e.g. `red`, `10px`, `flex`, ...

- each property has a defined set of allowed values (type) according to the **W3C CSS specification**

```
color: blue;
font-size: 12pt;
```

- values can be combinations of several basic types

```
margin: 10px 20px;
font: italic small-caps bold 16px/1.5 Arial;
```

Value Types

- **specific property values**
  - predefined textual values directly in the specification for particular properties
  - `display: block | inline | inline-block | flex | grid | …`
- **numbers**
  - integers and decimals
  - `line-height: 1.5`, `z-index: 10;`, …
- **lengths, sizes**
  - absolute: `px`, `cm`, `mm`, `in`, `pt`, `pc`
  - `1in` = `96px`, `1in` = `72pc`, `1pc` = `12pt`
  - relative: `em`, `rem`, `%`, `ex`, `ch`, `vw`, `vh`, `vmin`, `vmax`
  - `margin: 10px`, `font-size: 1.2em`, …
  - examples below…

Value Types: em, rem

```
html {
  font-size: 20px;
}

.em2 {
  font-size: 2em; /* 2x parent */
}

.rem2 {
  font-size: 2rem; /* 2x root */
}

body > div {
  border: 2px solid blue;
  margin: 2rem;
}

div > div {
  border: 2px solid green;
  margin: 2rem;
}
```

```
Default font size: 1rem
                                Font size: 2rem  F

                            Font size: 2rem  Fon


                            Font size: 2em   Fon
```

Value Types: vw, %

```
html { border: 2px solid red }
body { border: 2px solid black }

.pct75 {
  /* 75% of parent width */
  width: 75%;
}

.vw75 {
  /* 75% of viewport width */
  width: 75vw;
}

div {
  border: 2px solid blue;
  margin: 2rem;
}

div > div {
  border: 2px solid green;
}
```

75% 75vw
75%

75vw

Value Types: vmax

```
html {
  border: 2px solid red;
  height: 99vh;
}

.vmax50 {
  /* 50% of the larger viewport dimension */
  width: 50vmax;
}

div {
  border: 2px solid blue;
  margin: 2rem;
}
```

50vmax

Value Types

- **functional values**
  - special syntax with parameters
  - `calc()`, `var()`, `min()`, `max()`, `clamp()`, `attr()`
  - `width: calc(100% - 40px);`
- **URL and references**
  - link to a file or resource.
  - `background-image: url("bg.png"); cursor: url("cursor.cur"), auto;`
- **colors**
  - keywords (`red`, `blue`, `transparent`)
  - hex notation (`#ff0000`, `#f00`)
  - functions: `rgb()`, `rgba()`, `hsl()`, `hsla()`, `lab()`, `color()`

Value Types: named colors

Value Types: colors

Color:

Preview

- HEX:

Other Value Types (less used)…

- **time units**
  - s (seconds), ms (milliseconds)
  - animations and transitions: `transition-duration: 200ms;`
- **angle units**
  - used for transforms and gradients
  - `deg, grad, rad, turn`
- **frequency, resolution, etc.**
  - Hz, kHz (frequency – rarely used)
  - `dpi, dpcm, dppx` (resolution)

Variables

- user-defined properties that start with two dashes (`--`).
- store a value (e.g. color, size, number) and can be **reused** within a style rule
- the value is retrieved using the `var(--name)` function

```
:root {
  --main-color: #3498db;
  --padding: 1rem;
}

button {
  background-color: var(--main-color);
  padding: var(--padding);
}

button.danger {
  --main-color: #e74c3c; /* local override */
}
```

Properties

Introduction, principles

Property

- determines how selected elements are displayed (color, dimensions, layout, font, animation, etc.)
  - e.g. `color`, `font-size`, `margin`, `display`, ...
- the set of properties is defined in the **W3C CSS specification**

```
color: blue;
font-size: 12pt;
```

Types of Properties

- **typography and text** – affect font and text content
  - `font-family`, `font-size`, `font-weight`, `color`, `text-align`, `line-height`, ...
- **colors and background** – control background and color scheme
  - `background-color`, `background-image`, `background-position`, `opacity`, ...
- **box model (dimensions and margins)** – define size, outer and inner spacing, borders
  - `width`, `height`, `margin`, `padding`, `border`, `box-sizing`, ...
- **layout and positioning** – affect how elements are stacked and aligned
  - `display`, `position`, `float`, `z-index`, ...
  - Flexbox
  - CSS Grid

Types of Properties

- **lists and tables** – properties for special elements
  - `list-style-type`, `list-style-image`, `border-collapse`, `caption-side`, ...
- **transforms, transitions, and animations** – enable movement, effects, and interactivity
  - `transform`, `transition`, `animation`, ...
- **filters and effects**
  - `filter`, `backdrop-filter`, `mask`, ...
- **interactivity and cursor**
  - `cursor`, `pointer-events`, `user-select`, ...
- **content and generated elements** – add or change content generated by CSS
  - `content`, `quotes`, `counter-reset`, `counter-increment`, ...

Font Properties

- **font style – `font-style`**

  normal*italicoblique*

- **font weight – `font-weight`**

  normallighter**boldbolder**100**900**

- **font variant – `font-variant`**

  normalSMALL-CAPS

Font Size

- **font-size**

16px 32px 1rem 2rem 100% 200% small x-small xx-small medium large x-large xx-large

```
html {
  /* set size for 1rem */
  font-size: 16px;
}
```

Font Family

- **font-family**
- generic font families:

serifsans-serifmonotype

- installed fonts:

```
span.verdana {
  font-family: Verdana, Arial, sans-serif;
}

span.times {
  font-family: "Times New Roman", sans-serif;
}
```

VerdanaTimes New Roman

- always specify **alternatives** (last must be generic)
- if the name contains spaces, it must be in quotes

External Fonts

- Google Fonts
- import CSS with font definition in HTML:

```html
<head>
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?family=Ubuntu:ital,
  wght@0,300;0,400;0,500;0,700;1,300;1,400;1,500;1,700&display=swap" rel="stylesheet">
</head>
```

- use the font in CSS:

```css
span.ubuntu {
  font-family: "Ubuntu", sans-serif;
  /* according to font support */
  font-weight: 400;
  font-style: normal;
}
```

Ubuntu

Custom Font

- the **@font-face** rule allows you to load a custom font (e.g. from .woff2 or .ttf file) and use it in the document

```
@font-face {
  font-family: 'NewFont';
  src: url('newFontFile.woff') format('woff'),
     url('newFontFile.ttf') format('truetype'),
     url('newFontFile.svg') format('svg');
  font-weight: bold;
  font-style: italic;
  font-stretch: normal; /* font stretch */
}


h1 {
  font-family: 'NewFont', sans-serif;
  font-weight: bold;
  font-style: italic;
}
```

Text Properties

- **text decoration – `text-decoration`** (`-style`, `-color`, `-thickness`)

```
noneoverlineunderlineline-throughunderline red 6px
```

- **text transformation – `text-transform`**

```
noneCapitalizeUPPERCASElowercase
```

- **letter spacing – `letter-spacing`**

```
normal    .5rem
```

- **word spacing – `word-spacing`**

```
normal word spacing    .5rem word spacing
```

Text Alignment

- **horizontal alignment – `text-align`**
  - applies to block elements containing text (e.g. <p>)

```
leftcenterrightjustify
```

- **vertical alignment – `vertical-align`**

```
baselinesubsupertopbottom30%
```

Spacing

```
p {
  text-align: justify;
  line-height: 2;       /* line height */
  text-indent: 50px;      /* first line indent */
  overflow-wrap: break-word; /* break long words */
  word-break: break-word; /* break when needed */
  hyphens: auto;          /* hyphenation */
}
```

This paragraph shows how C
Notice the larger line hei
words are wrapped and may
SuperExtraMegaUltraLongWor

Text Columns

```
p {
  column-count: 2;
  column-width: auto;
  column-gap: 2rem;
  column-rule: 2px solid #ccc;
}
```

Multi-column layout in CSS allows long pi
into multiple vertical columns, similar t
This technique improves readability by sh
of content easier to scan. The browser ha
the content across columns and adapting t
the number of columns, the spacing betwee
especially useful for articles, blog post

Text Color and Shadow

- **color**: blue; **text-shadow**: offset-x offset-y blur-radius color;

```
p.simple {
  text-shadow: 4px 4px 10px black;
}

p.multiple {
  text-shadow:
  4px 4px 2px red,
  -4px -4px 2px blue;
}

p.neon {
  color: #0f0;
  text-shadow:
  0 0 5px #0f0,
  0 0 10px #0f0,
  0 0 20px #0f0,
  0 0 40px #0f0;
}
```

Simple shadowMulticolored shadowsNeon text

Background: Color, Image, Repeat

- **`background-color`**: `teal`; **`background-image`**: `url(…)`;
- **`background-repeat`**: `repeat` | `repeat-x` | `repeat-y` | `no-repeat`;

```
.color {
  background-color: teal;
}

.background {
  background-image: url("cat.jpg");
}

.repeat {
  background-image: url("cat.jpg");
  background-repeat: repeat-x;
}
```

background-colorbackground-imagebackground-

- default value of `background-repeat` is **`repeat`**

Background: Size, Position, Multiple Backgrounds

- **background-size**: auto | 200px | 50% 200px | cover | contain | …;
- **background-position**: 10px 20px | 50% 50% | top left | …;

```
.box {
  width: 100%;
  height: 600px;

  background-image:
    url('gandalf.jpg'),
    url('cat.jpg');

  background-repeat:
    no-repeat,
    no-repeat;
  background-size:
    200px,
    cover;
  background-position:
    top left,
    center;
}
```

Borders and Box Shadows

```
.box {
  border:
    10px solid red;
  border-radius:
    40px 20% 80px 10%;
  box-shadow:
    inset 0px 0px 50px yellow,
    0px 0px 50px red;
}
```

- **border**: thickness style color
- **border-radius**:
  - 20px;
  - TL TR BR BL;
- **box-shadow**: offset-x offset-y blur-radius spread-radius color inset

Display

- **`display`**:
  - `none` – element is not displayed at all (does not take up space)
  - `block` – block element (takes full width, starts on a new line)
  - `inline` – inline element (flows in line, cannot set width/height)
  - `inline-block` – inline, but can set width/height

- **positioning and layout creation...**
  - `table`, `inline-table`
  - `table-row`, `table-cell`, ...
  - `flex`
  - `grid`

Summary

links

Key Information

- concepts: style rule, cascade, rule, selector, property, value
- external, internal, and inline style rules
- W3C specification: https://www.w3.org/Style/CSS/
- validation: https://jigsaw.w3.org/css-validator/
  - it does not consider new CSS modules!

- examples, tutorials
  - https://developer.mozilla.org/en-US/docs/Web/CSS
  - https://www.w3schools.com/css/

To be continued…